



A) Sobre a entrada:

- 1) A entrada de seu programa deve ser lida da entrada padrão.
- 2) A entrada é composta por vários casos de teste, cada um descrito em um número de linhas que depende do problema.
- 3) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- 4) Cada linha, incluindo a última, contém o caractere final-de-linha.
- 5) O final da entrada coincide com o final do arquivo.

B) Sobre o seu código:

- 1) O nome do arquivo de seu programa é o nome do problema seguido da extensão da linguagem escolhida.
- 2) O seu programa deve ser escrito em uma das linguagens permitidas: Java, C, C++ ou Python 2 ou 3.
- 3) Um problema pode ser resolvido em uma linguagem diferente de outro, se desejar.
- 4) O seu programa deve ler a entrada do console (entrada padrão).
- 5) O seu programa deve gerar a saída em console (saída padrão).

C) Sobre a saída:

- 1) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- 2) Cada linha, incluindo a última, deve conter o caractere final-de-linha.

D) Sobre a competição:

- 1) Linguagens aceitas:
 - a) C, extensão “.c” → `gcc -std=c99 -O2 -lm`
 - b) C++, extensão “.cpp” → `g++ -std=c++17 -O2 -lm`
 - c) Java, extensão “.java” → `LC_ALL=en_US.UTF-8 java .. +2s` para limite de tempo
 - e) Python 3, extensão “.py” → `python3 .. +1s` para limite de tempo
- 2) As questões, embora escritas de forma original pela equipe organizadora, podem ser baseadas em problemas de mesma categoria já existentes.

Problema A

Agenda

Arquivo: *agenda*.*[c|cpp|java|py]*

Limite de tempo: 1 s

O Problema: Alice tem até 72 horas para organizar todos os seus compromissos e viajar. Desta forma, ela organiza todos os seus compromissos em uma agenda. Mas eventualmente precisa realocá-los ou cancelá-los.

Alice precisa de sua ajuda para gerenciar seus compromissos na sua agenda. Para cada compromisso, sabe-se sua descrição, o dia e hora de início e a duração em horas.

Você receberá um fluxo do agendamento e cancelamentos dos compromissos de Alice. Desta forma, você irá alocar e desalocar espaços na agenda de acordo com o fluxo recebido. A alocação deve respeitar as seguintes regras:

- Não existem compromissos sobrepostos;
- Não é possível dividir o compromisso, uma vez alocado, deve ser concluído, no tempo previsto;
- Nem sempre é possível alocar o compromisso na hora desejada/sugerida por Alice. Neste caso, a tarefa deve ser alocada no próximo horário que couber o compromisso; E caso não caiba, não é alocada.
- Os compromissos sempre têm duração e início em horas inteiras (minutos são desprezados).
- Durante o cancelamento, deve ser excluída todas as ocorrências daquele compromisso.

Você deverá ser capaz de informar Alice, qual a duração máxima que um compromisso pode ser agendado. A tabela exemplifica a agenda após a alocação dos compromissos:

- trabalho com duração de 10 horas, a partir de 15h do dia 1;
- ida ao mercado com duração de 5 horas a partir das 8h do dia 1.

DIA 1																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
								mercado							trabalho								

DIA 2																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
trabalho																							

DIA 3																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

Caso a Alice deseje agendar o compromisso:

- aula de violão com duração de 6 horas a partir das 8h do dia 1.

O agendamento ficaria como segue, na primeira oportunidade que couber a duração, após o horário de início sugerido.

DIA 1																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
								mercado							trabalho								

DIA 2																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
trabalho	aula_violao																						

DIA 3																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

Realizando outros agendamentos e cancelamentos, como segue:

- agendamento de aula de violão com duração de 6 horas a partir das 14h do dia 2;
- cancelamento da ida ao mercado;
- agendamento de trabalho por 10 horas a partir das 14h do dia 3.

DIA 1																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
															trabalho								
DIA 2																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
trabalho	aula_violao													aula_violao									
DIA 3																							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
														trabalho									

A duração máxima de um novo compromisso é 18h. Período compreendido entre 20h do dia 2 às 13h do dia 3.

Entrada:

A entrada consiste de uma linha contendo um inteiro N ($1 \leq N \leq 100$), indicando a quantidade de operações a serem realizadas na agenda de Alice. Para cada operação: Uma linha contendo ‘A’ para agendamento ou ‘C’ para cancelamento e uma sequência de no máximo 12 caracteres (ASCII sem acento) sem espaço indicando a descrição do compromisso.

Se o compromisso é um agendamento, uma linha contendo um número inteiro S ($1 \leq S \leq 72$) indicando a duração em horas; um número inteiro D ($1 \leq D \leq 3$) indicando o dia do compromisso e um número inteiro H ($0 \leq H < 24$) indicando a hora do dia do compromisso.

Saída: Um número inteiro indicando a duração máxima que um novo compromisso pode ser agendado.

Exemplo de entrada:

```
6
A trabalho
10 1 15
A mercado
5 1 8
A aula_violao
6 1 8
A aula_violao
6 2 14
C mercado
A trabalho
10 3 14
```

Saída do exemplo de entrada:

```
18
```

Problema B

Buscando o Melhor Caminho

Arquivo: *buscando*.*[c|cpp|java|py]*
Limite de tempo: 1 s

O Problema: Você e seus colegas motociclistas adoram estradas, a paixão é rodar em estradas. Mas não gostam nada de cidades, atravessar uma cidade representa ter de passar por lombadas, semáforos, ruas simples que não são estradas, e outros empecilhos colocados no caminho. Então na viagem se existem 2 caminhos para chegar ao mesmo destino, é preferível aquele que passe por menos cidades, mesmo que seja mais comprido. Também é proibitivo estradas de chão, estas são inclusive excluídas do mapa. Então dado um mapa de estradas transitáveis e cidades que atravessam (atravessar uma cidade é passar por seu trecho urbano) foi pedido a você que indique entre a origem e destino quantas cidades são passadas (excluem-se a origem e o destino).

Entrada:

A entrada consiste de vários casos de teste onde cada caso de teste é composto por um mapa e por um conjunto de roteiros. Um caso de teste começa com 2 números inteiros, o primeiro: N ($1 \leq N \leq 10$) o número de cidades no mapa, as cidades são numeradas de 1 a N , e M ($0 \leq M \leq N^2/2$) o número de trechos de estradas que ligam dois trechos urbanos de cidades distintas. Em seguida são M linhas indicando duas cidades que cujos trechos urbanos são ligados por uma estrada. As estradas não se cruzam, exceto nas cidades. Em seguida um número inteiro Q ($0 < Q \leq N$) o número de roteiros que queremos consultar. Nas Q linhas seguintes dois números indicando a cidade de origem e de destino respectivamente. A entrada termina com o fim dos casos de teste.

Saída: Para cada caso de teste Q linhas indicando o número de cidades passadas no roteiro, uma linha para cada consulta. Ou a palavra “IMPOSSIVEL” sem acento e sem aspas se não houver caminho entre as duas cidades. Entre cada caso de teste deve pular uma linha em branco

Exemplo de entrada:

```
6 6
5 3
2 4
3 6
1 2
1 3
6 1
2
6 2
5 4
2 0
1
1 2
2 1
2 1
1
1 2
```

Saída do exemplo de entrada:

```
1
3

IMPOSSIVEL

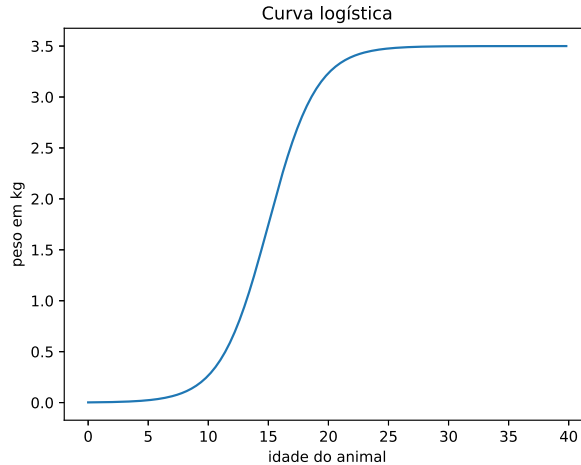
0
```

Problema C
Crescimento Animal
Arquivo: crescimento.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Jucimar é um tradicional criador de aves em sua região. Ao longo do tempo, ele percebeu que certas raças e gêneros do animal apresentam crescimento mais rápido que outros. Jucimar cria os animais desde o seu nascimento até completar 3.5 kg, quando a ave atinge esse peso, então ele a vende.

Jucimar precisa de sua ajuda para entender o comportamento de crescimento das aves, considerando essa diversidade de raça e gênero. Para isso, ele realiza pesagens diárias de sua ave. Ele sabe que as aves sempre crescem de 0.3kg a 3.5kg, no entanto, não sabe o quão rápido isso acontece. Ele também sabe que o crescimento de organismos na biologia tem uma curva em formato de S, como a curva de crescimento a seguir:



Essa curva pode ser descrita pela seguinte equação de crescimento:

$$f(x) = \frac{L}{1 + a^{-k(x-x_0)}}$$

Onde a = base da potência de crescimento: $a = 2.7183$

x_0 = valor de x no ponto médio da curva: $x_0 = 15$ dias

L = valor máximo da curva $L = 3.5$ kg

k = declividade da curva $[k] = \text{dias}^{-1}$.

Ainda, nesse contexto, x = idade do animal na pesagem

$f(x)$ = peso para o dia x em kg.

O valor de L , como visto, é conhecido pelo Jucimar. Empiricamente, ele também aprendeu o valor de x_0 , indicado acima, percebendo que a partir do 15º dia, o crescimento começa a desacelerar. Portanto, dados $L = 3.5$ e $x_0 = 15$, você deverá ajudá-lo a encontrar o valor de k , o parâmetro que indica o quão lento ou quão rápido esse crescimento ocorre.

Dadas as pesagens que Jucimar realizou em uma ave, você deverá descobrir o parâmetro k que melhor aproxima a curva logística aos dados de pesagem. Para isso, dada a entrada, deverá encontrar o k que resulta no menor Erro Absoluto Médio (MAE).

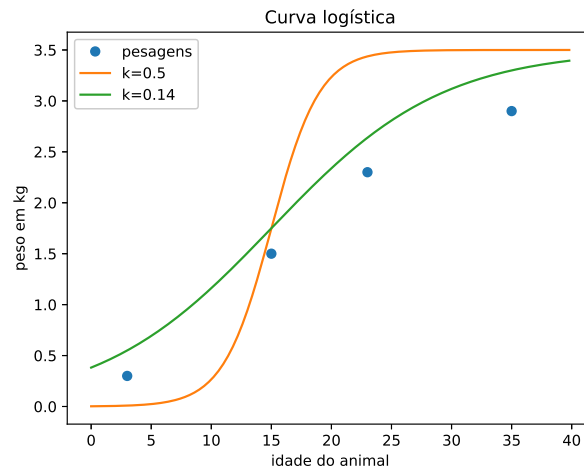
$$MAE(y, y') = \frac{1}{n} \sum_{i=1}^n |y_i - y'_i|$$

Onde n = o número de pesagens

y_i o peso do animal no dia i

y'_i o valor da curva logística para o dia i , isto é $f(i)$.

Veja um exemplo de pesagens realizadas, e erro médio para duas curvas (2 valores de k).



MAE para curva laranja = 0.57

MAE para curva verde = 0.31

Entrada:

Uma linha contendo o número de pesagens N ($4 \leq N \leq 100$). As N linhas seguintes contém um par: o número de dias de vida da ave e o seu peso naquele dia.

Saída: Valor de k , com precisão de 3 casas decimais, sabendo que k está no intervalo $[0.01, 1.0]$, indicando a velocidade de crescimento da ave pesada que retorne o menor **MAE** para a curva de crescimento.

Exemplo de entrada:

```
4
3 0.3
15 1.5
23 2.3
35 2.9
```

Saída do exemplo de entrada:

```
0.081
```

Problema D
Duplas Escolhas
Arquivo: duplas.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Como sempre, há um sorteio para escolha de time. Normalmente o cabeça de cada time escolhe, na sua vez, aquele que ele acha o melhor dentre o conjunto de escolha. Mas para complicar um pouco seu amigo surgiu com a ideia de duplas escolhas, ou seja a cada vez, o cabeça deve escolher uma dupla, assim, dentre o conjunto perfilado, a dupla deve ser de dois vizinhos. Para se sair bem, seu amigo pediu que você fizesse um algoritmo para definir as escolhas dele. Então estão $N : N|4 \equiv 0$ pessoas perfiladas, e ele lhe indicou o nível de qualidade de cada um, de forma a maximizar a capacidade de seu time. Suponha o conjunto de pessoas e escolhas:

Primeiro uma escolha simples, na ordem dos candidatos

Amigo	3 6 1 8 4 5 6 3 1 2 3 4 5 6 7 8	Adversário
1 2 5 6 Total = 18		3 4 7 8 Total = 18

Uma escolha mais cuidadosa:

Amigo	3 6 1 8 4 5 6 3 1 2 3 4 5 6 7 8	Adversário
4 5 1 2 Total = 21		6 7 3 8 Total = 15

Então é importante analisar com cuidado os pares a serem escolhidos para sair em vantagem. A vantagem de seu amigo é que ele tem uma moeda viciada (2 caras) e sempre ganha no cara e coroa na hora de escolher primeiro o time.

Entrada:

A entrada consistem em várias linhas, cada linha é um caso de teste. Cada linha é composta por um número inteiro positivo N ($0 < N \leq 1.000$) e $N|4 \equiv 0$, representando quantos jogadores estão perfilados, e em sequência N valores inteiros positivos v_i ($0 \leq v_i \leq 1.000$) representando a qualidade individual de cada jogador na fila. Os casos de teste terminam com o fim da entrada.

Saída:

Para cada caso de teste, em uma linha, um número único indicando qual a máxima soma de qualidade que o time de seu amigo pode atingir.

Exemplo de Entrada:

```
8 3 6 1 8 4 5 6 3
4 1 2 3 4
4 1 3 2 4
4 3 2 1 4
```

Saída do exemplo de Entrada:

```
21
7
6
5
```


Problema E
Escopo de variável
Arquivo: escopo.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Você, como um cientista da computação entusiasta está desenvolvendo sua própria linguagem de programação, e a batizou de Oxente. Você já definiu seus tipos de dados, palavras reservadas da linguagem e regras de uso da mesma. Agora, precisa avaliar a semântica dos seus códigos escritos em Oxente. Uma das etapas semânticas é avaliar o escopo das variáveis. Isto é, definir e checar o acesso e visibilidade de uma variável dentro de cada bloco do código.

Seja o seguinte código fonte **exemplo.oxe** na linguagem Oxente:

```
1: [ #bloco1
2:     int a;
3:     a=10;
4:     [ #bloco2
5:         int b;
6:         b=a+20;
7:     ] #fim bloco2
8:     print(b);
9:] #fim bloco1
```

Nesse código, assumimos que a variável **b** é acessível apenas no bloco 2, onde foi declarada. Portanto a chamada **print(b)** é incorreta, pois **b** não é acessível naquele escopo. Enquanto que a variável **a** é acessível no bloco 1 e bloco 2 (pois este está contido no bloco 1).

É possível perceber que Oxente é uma linguagem bem sucinta, onde os blocos são definidos por **[]** para abertura e fechamento, e as variáveis são compostas apenas de um caractere minúsculo.

Desta forma, você deverá analisar o escopo, informando se o código está semanticamente correto ou se existe acesso a variável não visível naquele escopo.

Entrada:

Uma única linha, com no máximo 1000 caracteres, contendo uma versão simplificada de um código fonte em Oxente. A linha contém uma sequência de caracteres sem espaço onde **[]** indicam abertura e fechamento de escopo, **Dx** indica declaração da variável **x** e **Ax** indica um acesso à variável **x**. Para o exemplo.oxe, mostrado anteriormente, a versão simplificada é: **[DaAa[DbAaAb]Ab]**. Onde:

- Da corresponde à declaração da variável **a** na linha 2: **int a;**
- Aa corresponde ao acesso à variável **a** na linha 3: **a=10;**
- Db corresponde à declaração da variável **b** na linha 5: **int b;**
- Aa e Ab correspondem ao acesso às variáveis **a** (leitura) e **b** (atribuição) na linha 6: **b=a+20;**
- Ab corresponde ao acesso à variável **b** na linha 8: **print(b);**

Saída: Uma única linha contendo “Semanticamente correto!” ou “Erro de escopo!” decorrente do resultado da análise.

Exemplo de entrada:

Saída do Exemplo de entrada:

[DaAa[DbAaAb]Ab]

Erro de escopo!

[DaAa[DbAaAb]DbAb]

Semanticamente correto!

[DaAa[DbAaAb] [DcAa]]

Semanticamente correto!

Problema F
Fila Justa
Arquivo: fila.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Sabemos que bancos, correios e outras instituições de atendimento público de grande movimento utilizam senhas para acesso ao atendimento, cada senha tem um ID único (iniciando em 1 no começo dos movimentos) crescente. Mas é importante lembrar que existem usuários que merecem atendimento prioritário, por lei, então além do ID a senha carrega a informação entre usuários com prioridade “P” e comuns “C”. Num modelo de senhas é possível criar uma fila única dando prioridade a quem merece mas também atendendo de forma justa os demais usuários. Foi definida que a cada 3 usuários de prioridade atendidos (se houverem) um usuário comum deve ser atendido. Além disto, o atendimento deve ser por ordem de chegada, ou seja, uma fila, um usuário é inserido no final da fila, os usuários com prioridade podem ultrapassar usuários comuns desde que respeitem as regras abaixo:

- Um usuário com prioridade não pode ser inserido como primeiro da fila, exceto em fila vazia.
- Um usuário com prioridade não pode ser inserido como segundo da fila, se o primeiro já for usuário com prioridade, exceto em fila com um único usuário.
- Um usuário com prioridade não pode ser inserido como terceiro da fila, se os dois primeiros forem usuários com prioridade, exceto em fila com apenas dois usuários.
- Um usuário com prioridade deve ser inserido na fila respeitando o critério de atendimento: 3 com prioridade e 1 comum.

Então a você foi pedido um algoritmo para criar uma fila que respeite o atendimento de prioridade.

Entrada:

A entrada contém vários casos de teste, cada caso de teste começa com um número N de movimentos ($0 < N < 100.000$), cada movimento em uma linha possui um caractere indicando se o usuário está entrando na fila “E” ou saindo da fila para ser atendido “D”. Não vamos considerar usuários desistindo de ser atendido. Quando um usuário entra na fila outro caractere, separado por um espaço, que indica se o usuário é comum “C” ou de prioridade “P”. É garantido que somente há movimento de saída quando houver usuário na fila. O fim dos movimentos coincide com o fechamento da agência, neste caso não haverá mais movimentos de entradas e subentende-se que os movimentos a seguir são apenas de saída até o final da fila. Os casos de teste terminam com $N = 0$.

Saída: Uma única linha para cada caso de teste contendo a ordem dos IDs das senhas, cada ID separado do outro por um espaço em branco, correspondente à ordem em que houve o atendimento.

Exemplo de entrada:

```
9
E C
E P
E P
E C
D
E P
D
E C
E P
5
E P
E C
E P
E C
E P
0
```

Saída do exemplo de entrada:

```
1 2 3 4 5 7 6
1 2 3 5 4
```

Problema G
Gastando menos
Arquivo: gasto.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Atualmente precisamos ser bem espertos ao fazer compras no supermercado. A indústria nos vende a ideia de que as maiores embalagens são mais econômicas, comprando mais produto por menor preço. Mas na prática nem sempre acontece assim.

Você acredita que gastar menos não é sobre pagar o valor mais barato de um produto, mas sim, pagar o valor do produto que tem melhor custo benefício. O melhor custo benefício é sempre aquele produto no qual o valor unitário é o de menor custo, ou seja, o valor de 1 kg é o mais barato.

Desta forma, dado 2 produtos de uma mesma marca e com embalagens de tamanhos diferentes (pesos diferentes), você deverá informar qual produto tem melhor custo benefício.

Entrada:

A entrada consiste de uma única linha composta por 4 números com ponto flutuante e precisão de 2 casas decimais. Os números representam, respectivamente: peso do primeiro produto, valor do primeiro produto, peso do segundo produto, valor do segundo produto. Onde peso é dado em kg, e valor em reais.

Saída: A saída deve ser 'primeiro' ou 'segundo', indicando produto de melhor custo benefício. Em caso de empate, a saída deve ser 'ambos'.

Exemplo de entrada:

Saída do Exemplo de entrada:

3.50 10.50 1.20 2.09

segundo

0.80 5.00 0.40 2.50

ambos

10.00 400.00 2.50 140.00

primeiro

Problema H

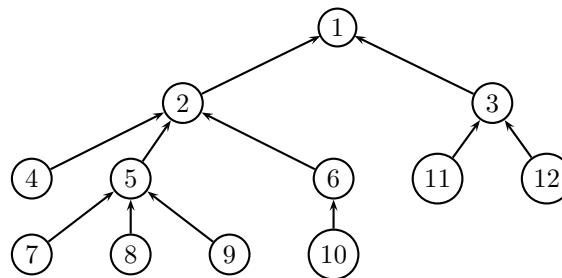
Hierarquia

Arquivo: *hierarquia*.*[c|cpp|java|py]*

Limite de tempo: 1 s

O Problema:

A Companhia de Paisagismo Urbano (CPU) cresceu e como toda grande empresa, tem problemas com seus funcionários. Então a ordem da casa é: *Respeite a Hierarquia*. A hierarquia é na forma de chefe e subordinado, a empresa possui a presidência (na forma de uma única pessoa), abaixo da presidência vem a diretoria de cada unidade. Abaixo da diretoria, a chefia dos departamentos, abaixo as gerências, e assim sucessivamente. Acontece que além de apresentar uma hierarquia, a empresa, para uma melhor economia, é bem departamentalizada, as unidades distintas às vezes cuidam só de aspectos específicos do paisagismo, por exemplo, uma unidade cuida da instalação de fontes, enquanto que outra da iluminação, e para se ter fontes iluminadas precisa do serviço das duas unidades, e é aí que acontecem os problemas, pois um funcionário às vezes se vê em querelas com outro. Por isso o mote: *Respeite a Hierarquia*, quem deve resolver é o chefe imediato mais próximo, aquele chefe na hierarquia da empresa que é chefe indireto de ambos, o primeiro. Claro na presidência poderia ser o âmbito para resolver as pendengas, mas se houver um nível inferior responsável por ambos funcionários então é neste nível que será realizada esta tarefa difícil de conciliação (aliás nenhuma chefia quer esta tarefa, cabe mesmo àquela que é a primeira chefia direta ou indireta de ambos. Veja a estrutura abaixo, cada funcionário indicado por seu ID na hierarquia, Na presidência sempre está a número 1.



Por exemplo, na estrutura hierárquica acima, se a diferença for entre os funcionários 7 e 9, então a chefia 5 deve resolver, mas se for entre 8 e 10 cabe à chefia 2. Sempre pode sobrar para o presidente, por exemplo se o problema for entre 4 e 12. Como a estrutura é grande, a você foi pedido que dado os dois funcionarios futriquentos, indique qual a chefia para resolver.

Entrada: O caso de teste começa com uma linha indicando o número inteiro N ($1 < N < 300$) de funcionários da empresa. Na linha seguinte, são $N - 1$ números onde o i -ésimo número indica quem é o chefe imediato do funcionário $i+1$. Já que o presidente é o número 1, o primeiro número indica o chefe imediato do funcionário 2, o segundo do funcionário 3 e assim sucessivamente. Na próxima linha o número inteiro Q ($0 < Q < N/2$) representa o número de querelas a resolver e em seguida são Q linhas contendo 2 números representando o ID de cada funcionário.

Saída:

São Q linhas contendo em cada linha o ID da chefia responsável por resolver a querela.

Exemplo de entrada:

```
12
1 1 2 2 2 5 5 5 6 3 3
3
7 9
8 10
4 12
```

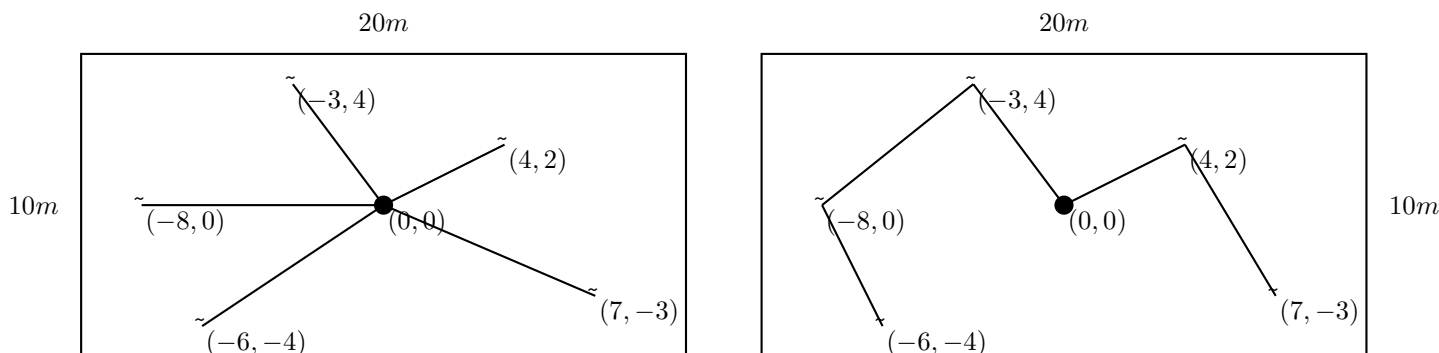
Saída do exemplo de entrada:

```
5
2
1
```

Problema I
Iluminando o Jardim
Arquivo: *iluminando.[c|cpp|java|py]*
Limite de tempo: 1 s

O Problema:

A Companhia de Paisagismo Urbano (CPU) ganhou uma concorrência para criar a iluminação em um jardim público. O jardim possui várias árvores e o objetivo é iluminar, de baixo para cima, as árvores do jardim. O problema é que o custo dos cabos a serem enterrados não é pouco então queremos minimizar os custos. O jardim é quadrado e no centro há uma torre que recebe a iluminação pública para distribuir às árvores. Veja um exemplo abaixo, considerando uma ligação onde cada árvore está ligado à torre e outra onde há um gasto menor de cabos e todas as árvores estão ligadas:



Realmente, na primeira ligação são gastos 32,30m de cabo, enquanto que na segunda são gastos apenas 26,18m de cabo. A você foi pedido um algoritmo no qual, dada a disposição das árvores no jardim, indique qual a metragem mínima necessária de cabo para realizar a iluminação nas árvores.

Entrada: São vários casos de teste, cada caso de teste começa com uma linha com três valores inteiros L , C e A , onde L e C ($1 \leq L, C \leq 100$) é o comprimento e largura, respectivamente, do jardim em metros. A , ($0 < A \leq 10$) o número de árvores no Jardim. Em seguida são A linhas onde em cada linha está a posição da árvore no Jardim representada por dois números de ponto flutuante a partir de uma orientação em relação às bordas de largura e comprimento tendo a torre central como origem na posição (0,0). Os casos de teste terminam com $C = L = A = 0$

Saída:

O comprimento mínimo de cabos para ligar todas árvores à torre em metros com duas casas decimais.

Exemplo de entrada:

```
10 20 5
-3.00 4.00
-8.00 0.00
-6.00 -4.00
4.00 2.00
7.00 -3.00
0 0 0
```

Saída do exemplo de entrada:

26.18

Problema J
Jornada Suave
Arquivo: jornada.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Voltando à diversão, você e seus colegas motociclistas perceberam que não é só as cidades que é bom evitar, pois tem muita estrada, que mesmo asfaltada, representa um transtorno para os motociclistas. Por exemplo, a estrada que liga Canavieiras na BA à cidade vizinha Santa Luzia, é uma estrada com tantos buracos, que tem buraco no acostamento esperando a vez de entrar na estrada. Assim, no mapa rodoviário, vocês deram pontuação à dificuldade de passar pela estrada. Quanto maior a pontuação, mais difícil é trafegar. Assim às vezes vale a pena rodar centenas de quilômetros em estradas suaves a rodar dezenas em estradas precárias. Principalmente pelo fato que rodar é o principal objetivo. Então o mapa contém as estradas pontuadas em cada trecho entre dois cruzamentos. A você foi pedido que, com base no mapa com estradas pontuadas, escolha uma rota entre uma origem e destino (estes são sempre cruzamentos de estradas), dentre todas as rotas possíveis, de forma que a rota contenha as estradas em que o trecho de maior pontuação seja o mínimo possível.

Entrada: O caso de teste começa com dois números inteiro N ($0 < N \leq 20$) o número de interseções das estradas no mapa, e M ($0 \leq M \leq N^2/2$) o número de trecho de estradas no mapa entre as interseções. A seguir são M linhas contendo três valores inteiros: A, B, P , onde A e B são duas interseções ligadas por um trecho de estrada ($0 < A, B \leq N$) e P a pontuação do trecho ($0 \leq P \leq 1000$). Por fim um valor inteiro J ($0 < J \leq N/2$) representando as jornadas desejadas baseadas no mapa, e em seguida J linhas com dois valores inteiros O e D ($0 < O, D \leq N$) representando os pontos de interseção de origem e de destino da jornada.

Saída:

J linhas com o valor na jornada do trecho de maior pontuação, sendo este o menor possível com base no mapa. Ou a palavra "IMPOSSIVEL" se não houver rota entre os dois pontos de interseção escolhidos.

Exemplo de entrada:

```
6 8
5 3 3
2 4 1
5 1 2
3 6 2
1 2 3
4 6 1
1 3 1
6 1 1
2
6 2
5 4
2 0
1
1 2
2 1
2 1 2
1
1 2
```

Saída do exemplo de entrada:

```
1
2

IMPOSSIVEL

2
```

Problema K
Karaokê Animado
Arquivo: karaoke.[c|cpp|java|py]
Limite de tempo: 1 s

O Problema:

Dado o grande número de querelas existentes entre funcionários da CPU (Companhia de Paisagismo Urbano), o presidente percebeu que as diferenças aconteciam pois os funcionários não se conheciam ou pouco mantinham relações de amizade. Então resolveu fazer uma grande integração em uma festa com competição de talentos musicais no karaokê. Porém ele precisava de uma forma de medir as disputas e identificar quem ganhou ou quem perdeu. Sabe-se que as notas são representadas pelas letras, sendo:

Nota	Dó	Ré	Mi	Fá	Sol	Lá	Si
Representação	C	D	E	F	G	A	B

A distância entre as notas é chamado de Tom, mas existem dois pares de notas cuja distância é Meio Tom. Entre Mi e Fá, e entre Si e Dó. Também existem as representações indicando notas subindo ou descendo meio tom:

- Subindo meio tom (sustenido): Dó#, Ré#, Mi#, Fá#, Sol#, Lá# e Si#.
- Descendo meio tom (bemol): Dób, Réb, Mib, Fáb, Solb, Láb e Sib.

Com isto algumas notas possui mais de uma representação, na tabela abaixo, na mesma coluna estão as representações distintas da mesma nota, entre uma coluna e outra tem a distância de meio tom, inclusive entre a última e a primeira coluna:

C		D		E	F		G		A		B
B#	C#		D#		E#	F#		G#		A#	
	Db		Eb	Fb		Gb		Ab		Bb	Cb

Assim a ideia geral ao medir a participação de cada um no karaokê é aquele com a menor distância entre a reprodução e à versão original. Foi pedido a você um algoritmo que dadas as representações das duas versões: reprodução e original, que se pontuassem a menor distância entre elas. Por exemplo se a versão original era *E* e foi cantado *G*, a menor distância é 1.5 tons. Já entre *E* e *B*, a menor distância é 2.5 tons.

Entrada:

A entrada consiste de um número N em uma linha ($0 < N \leq 1000$) indicando o número de notas existentes na música. Aí seguem duas linhas onde em cada linha existem N notas da música na versão original e na versão cantada usando-se a representação acima indicado. Cada nota é separada uma da outra por um espaço em branco.

Saída:

Na saída um número único indicando a soma das menores distâncias entre cada par respectivo de notas com uma casa decimal.

Exemplo de entrada:

Saída do Exemplo de entrada:

5
C D Eb F B
C D# F Gb Cb

2.0

6
F D# G Ab G F
G E A# B A# G

7.0